Quantum Computing
Professor Gopal Pandurangan

Scribe Author: Armaan Amatya

Contents

Lecture of August 25, 2026 **3**

1 Efficient Algorithms and Finding Prime Numbers **3**
1.1 Randomized Algorithms 3

2 Random Sampling **3**
2.1 Sampling a Random Element of an Array 3
2.2 With and Without Replacement 4

3 Factoring **4**

4 Search **5**
4.1 Lower Bound for Deterministic Algorithms 5
4.2 Randomized and Quantum Algorithms 5

5 Probability **6**

Lecture of September 1, 2026 **8**

6 Query Complexity for Search **8**
6.1 Monte Carlo and Las Vegas Algorithms 8
6.2 Random Sampling 8

7 Coupon Collector Problem **9**

8 High Probability Bound **10**

9 Lower Bound for Randomized Algorithms **11**

10 Chernoff Bounds **11**

Lecture of September 3, 2026 **13**

11 Markov's Inequality **13**

12 Chebyshev's Inequality **14**

13 Chernoff Bound **14**

14 Search **15**

15 Birthday Paradox **15**
15.1 Expected Number of Collisions 16

16 Quantum Birthday Paradox **17**

Lecture of September 24, 2026 **18**

17 Deutsch's Algorithm **18**
17.1 The Quantum Query U_f 18
17.2 The Circuit 18

18 Phase Flip **19**

19 The n-Qubit Hadamard Transform **20**

20 Deutsch–Jozsa Algorithm **20**
20.1 Classical Query Complexity 21
20.2 The Algorithm 21

Lecture Date: August 25, 2026

Lecture Topic: Efficient Algorithms, Random Sampling, Lower Bound for Deterministic Search, Probability Basics

In this first lecture, we see why randomness and quantum computation matter for two basic problems, primality/factoring and search. We introduce random sampling as the basic primitive of randomized algorithms, prove a lower bound of $n-1$ queries for deterministic search, and review the basics of probability that we need for the rest of the course.

1 Efficient Algorithms and Finding Prime Numbers

Classical computing was deterministic until relatively recently, when randomness was introduced into algorithms. The next step, and the subject of this course, is quantum computing.

Consider the problem of deciding whether a given number n is prime. The input is the binary representation of n, so the *input size* is the number of bits

$$N = \log_2 n,$$

and not n itself.

Definition 1 ▶ Efficient Algorithm

An algorithm is **efficient** if it runs in time polynomial in the input size, i.e., in time $\mathcal{O}\left(N^k\right)$ for some constant k. For primality testing this means time $\mathcal{O}\left((\log_2 n)^k\right)$.

The trivial algorithm, trial division by all numbers up to $\sqrt{n}$, takes $\Theta(\sqrt{n}) = \Theta\left(2^{N/2}\right)$ time, which is exponential in the input size. For a long time no efficient algorithm was known. A deterministic polynomial-time algorithm was eventually found (running in time $\tilde{\mathcal{O}}(N^{12})$, later improved to $\tilde{\mathcal{O}}(N^6)$), but it is not used in practice, both because it is not particularly fast and because there is a better alternative.

1.1 Randomized Algorithms

The algorithm used in practice is *randomized*: the Miller–Rabin test decides primality with high probability in $\mathcal{O}(N^3)$ time. The idea is that if n is composite, then at least $3/4$ of the numbers in $\{2, \ldots, n-2\}$ are *witnesses* of the fact that n is composite, while a prime has no witness. Sampling one random number therefore detects a composite n with probability at least $3/4$, and after k independent samples a composite n goes undetected with probability at most $(1/4)^k$. If the test says "composite" it is always right; if it says "prime" it is wrong with probability at most $(1/4)^k$, which can be made as small as we like by increasing k.

2 Random Sampling

All that a randomized algorithm needs is the ability to *sample*. We assume that flipping a (fair) coin is an $\mathcal{O}(1)$ time primitive.

2.1 Sampling a Random Element of an Array

Consider an array with n slots (Figure 1) and assume for simplicity that n is a power of 2. Every index is a bit string of $\log_2 n = \log n$ bits, so we can sample a uniformly random element

by flipping one coin for each bit of the index. This takes $\mathcal{O}(\log n)$ time, and every index i is obtained with probability

$$\mathbb{P}(\text{index } i \text{ is chosen}) = \left(\frac{1}{2}\right)^{\log n} = \frac{1}{2^{\log n}} = \frac{1}{n}.$$

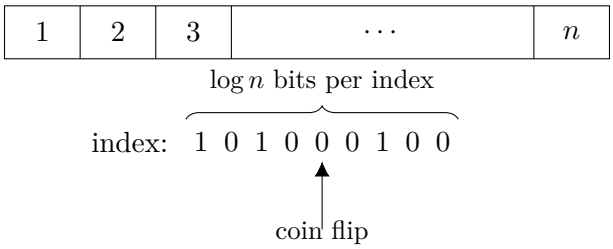


Figure 1: Sampling a random slot of an array with n slots: each of the $\log n$ bits of the index is the result of one coin flip.

We usually abstract away the $\mathcal{O}(\log n)$ and assume that an element can be sampled in constant time. This is only justified when the space we sample from has polynomial size (as a function of n), since then the number of coin flips is $\mathcal{O}(\log n)$; for a larger space the sampling time is no longer negligible.

2.2 With and Without Replacement

Random sampling can be done *with* or *without* replacement. We will almost always analyze sampling with replacement, because the analysis without replacement is more complex (the samples are no longer independent).

Suppose that we are looking for one particular element of the array. For every sample,

$$\mathbb{P}(\text{element} = \text{sample}) = \frac{1}{n}.$$

If we keep sampling (with replacement) until the element is found, then the expected number of samples, i.e., the number of steps of the algorithm, is

$$\mathbb{E}[\text{samples}] = \frac{1}{\mathbb{P}(\text{element} = \text{sample})} = n.$$

We will prove this in the next lecture, when we introduce the expectation and the geometric random variable. For now we can already compute the probability of *not* finding the element: a sample misses it with probability $1 - \frac{1}{n}$, and since the samples are independent,

$$\mathbb{P}(\text{not found after } k \text{ samples}) = \left(1 - \frac{1}{n}\right)^k, \qquad \mathbb{P}(\text{found within } k \text{ samples}) = 1 - \left(1 - \frac{1}{n}\right)^k.$$

3 Factoring

Factoring is the second motivating problem: given $n = pq$ with p, q prime, find p and q.

- Factoring is an *encryption primitive*: the security of widely used encryption schemes rests on it being hard. If factoring can be solved quickly, then the encryption is broken.
- No polynomial-time (in $N = \log n$) classical algorithm is known.
- There is a quantum algorithm that runs in polynomial time, *Shor's factoring algorithm* (time $\mathcal{O}(N^3)$).

- There is *no* known quantum algorithm that solves NP-complete problems in polynomial time.

Recall that NP is the class of problems whose solutions are easy to verify (in polynomial time), and that the NP-complete problems are the NP problems to which every NP problem reduces (equivalently, the problems that are reducible to 3-SAT and vice versa). Whether P = NP, i.e., whether NP-complete problems can be solved in polynomial time, is a famous open problem. A quantum computer can simulate a classical one with polynomial overhead, so if P = NP then quantum computers also solve NP-complete problems in polynomial time; conversely, if one could show that quantum computers cannot solve NP-complete problems in polynomial time, then P $\neq$ NP.

4 Search

The third problem, which we will study in detail, is *search*: we are given an (unsorted) array A with n slots, which can only be accessed through *queries* ("what is $A[i]$?"), and we want to find the index of a given element x. If the array were sorted, binary search would take $\mathcal{O}(\log n)$ queries; since it is not, linear search makes $\mathcal{O}(n)$ queries. Can we do better?

4.1 Lower Bound for Deterministic Algorithms

An upper bound of $\mathcal{O}(f(n))$ queries only needs one algorithm; a lower bound of $\Omega(f(n))$ has to hold for *all* algorithms. The standard tool for this is an *adversary argument*: given any algorithm, the adversary constructs an input on which the algorithm performs badly.

Theorem 1 ▶ Lower Bound for Deterministic Search

Every deterministic algorithm for search makes at least $n - 1$ queries in the worst case.

Proof. Fix a deterministic algorithm. The adversary does not commit to the input in advance; instead it answers the queries of the algorithm as they come. Whenever the algorithm queries a slot i, the adversary answers that $A[i] \neq x$ (and fills $A[i]$ with some other value).

Since the algorithm is deterministic, its kth query is determined by the answers to the first $k-1$ queries, so this defines one input for the algorithm. Suppose that the algorithm stops after $k \leqslant n-2$ queries and outputs an index j. At least $n-k \geqslant 2$ slots have not been queried, so the adversary can place x in an unqueried slot different from j. The resulting array is consistent with all the answers that were given, and the algorithm is wrong on it. Hence a correct algorithm makes at least $n-1$ queries. □

After $n-1$ queries that were all answered "no", only one slot is left and the algorithm knows that x is there without querying it, so the bound $n-1$ is tight.

4.2 Randomized and Quantum Algorithms

- The adversary argument breaks down for *randomized* algorithms: the queries are no longer determined by the previous answers, so the adversary cannot predict them. Nevertheless, randomized search needs $\Omega(n)$ expected queries. The proof uses *Yao's min-max principle*, which is a good general tool for proving lower bounds for randomized algorithms (next lecture).
- Grover's quantum algorithm solves search with $\mathcal{O}(\sqrt{n})$ queries (later in the course).

To show a *quantum advantage* (often called quantum supremacy in class) for a problem, we have to show that the problem needs $\Omega(f(n))$ queries classically but only $o(f(n))$ queries quantumly. Since randomized algorithms are the strongest classical algorithms, this means that we need lower bounds for *randomized* algorithms, not only for deterministic ones. This is why lower bounds for randomized algorithms play a bigger role in this course than lower bounds for deterministic algorithms.

5 Probability

We review the basic notions of (discrete) probability (Figure 2).

Definition 2 ▶ Sample Space, Probability Function, Events

A **sample space** S is a finite set of *simple events*; we write $|S| = n$. A **probability function** is a function

$$\mathbb{P}\colon S \to [0,1] \subseteq \mathbb{R}.$$

An **event** is a subset $A \subseteq S$, and its probability is

$$\mathbb{P}(A) = \sum_{a \in A} \mathbb{P}(a).$$

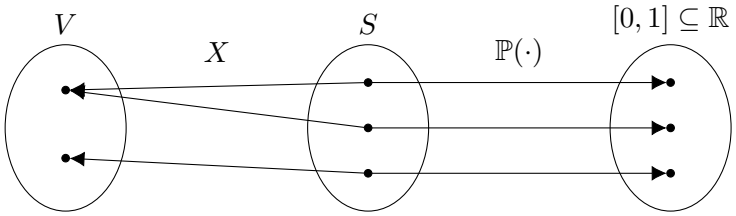


Figure 2: A sample space S of simple events. The probability function $\mathbb{P}(\cdot)$ maps every simple event to a number in $[0,1]$, and a random variable X maps every simple event to a value in V.

The probability function satisfies the following *axioms*:

1. $\mathbb{P}(A) \geqslant 0$ for every event A;
2. $\mathbb{P}(S) = 1$;
3. if $A_1, A_2, \ldots, A_n$ are pairwise disjoint events, i.e., $A_i \cap A_j = \emptyset$ for all $i \neq j$, then

$$\mathbb{P}(A_1 \cup A_2 \cup \cdots \cup A_n) = \sum_{i=1}^{n} \mathbb{P}(A_i).$$

Definition 3 ▶ Random Variable

A **random variable** is a function

$$X\colon S \to V$$

from the sample space to a set of values V (usually $V \subseteq \mathbb{R}$). For a value $r \in V$,

$$\mathbb{P}(X = r) = \sum_{s \in S\,:\,X(s)=r} \mathbb{P}(s).$$

For example, in random sampling the sample space S is the set of the n possible outcomes of the $\log n$ coin flips, and the random variable X is the index that is sampled. Every $s \in S$ has $\mathbb{P}(s) = 1/n$, and $\mathbb{P}(X = i) = 1/n$ for every index i. In the next lecture we define the expectation $\mathbb{E}[X]$ of a random variable and prove that the expected number of samples until a given element is found is n.

Lecture Date: September 1, 2026

Lecture Topic: Query Complexity of Search, Coupon Collector Problem, Lower Bound for Randomized Algorithms, Chernoff Bounds

In this lecture, we study the query complexity of search for classical randomized algorithms. We analyze random sampling, relate it to the coupon collector problem, state a lower bound for randomized search algorithms using Yao's minmax principle, and introduce Chernoff bounds.

6 Query Complexity for Search

We are given an array of n slots (Figure 3). The array can only be accessed through *queries*: a query to a slot x returns the value $f(x)$. In the *search* problem, the goal is to find the slot x such that

$$f(x) = 1.$$

The cost of an algorithm is its *query complexity*, i.e., the number of queries that it makes.

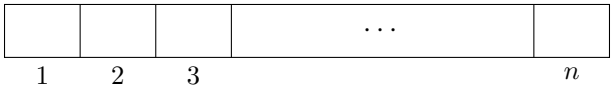


Figure 3: An array of n slots. We want to find the slot x with $f(x) = 1$.

6.1 Monte Carlo and Las Vegas Algorithms

Definition 4 ▶ Monte Carlo and Las Vegas Algorithms

A **Monte Carlo** algorithm is a randomized algorithm whose error probability is > 0.
A **Las Vegas** algorithm is a randomized algorithm whose error probability is $= 0$.

6.2 Random Sampling

Consider random sampling *with replacement*: in every step we query a slot that is chosen uniformly at random among the n slots. For every query,

$$\mathbb{P}(\text{Success}) = \frac{1}{n}.$$

The number of queries until the first success is a *geometric* random variable with parameter $1/n$. Hence

$$\mathbb{E}[\text{queries}] = n.$$

If random sampling is stopped after a fixed number of queries, then it is a Monte Carlo algorithm, since it can fail to find the slot. If we keep sampling until the slot is found, then it is a Las Vegas algorithm, and

$$\mathbb{E}[\text{queries until find}] = n.$$

7 Coupon Collector Problem

Suppose that we want to distinguish 2 binary arrays (Figure 4). To do this with random sampling, all the n slots have to be accessed. The number of queries needed by random sampling (with replacement) to access all the n slots is the *coupon collector problem*. We show that

$$\mathbb{E}[\text{queries}] \leqslant \mathcal{O}(n \log n) \quad \text{and} \quad \mathbb{E}[\text{queries}] \geqslant \Omega(n \log n),$$

i.e., $\mathbb{E}[\text{queries}] = \Theta(n \log n)$.

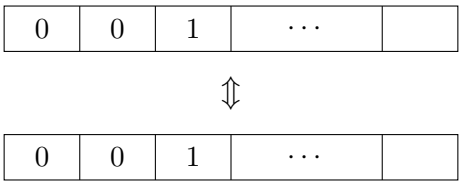


Figure 4: To distinguish 2 binary arrays.

Theorem 2 ▶ Coupon Collector

Let X be the number of queries made by random sampling (with replacement) until all the n slots are accessed. Then

$$\mathbb{E}[X] = nH_n = \Theta(n \log n),$$

where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ is the nth harmonic number.

Proof. Write

$$X = X_1 + X_2 + \cdots + X_n,$$

where X_i is the number of queries made to access a new slot, after $i-1$ different slots have already been accessed. When $i-1$ slots have been accessed, a query accesses a new slot with probability $\frac{n-(i-1)}{n}$. Hence X_i is a geometric random variable,

$$X_i \sim G\left(\frac{n-(i-1)}{n}\right), \qquad \mathbb{E}[X_i] = \frac{n}{n-(i-1)}.$$

By linearity of expectation,

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E}[X_1 + X_2 + \cdots + X_n] \\
&= \sum_{i=1}^{n} \mathbb{E}[X_i] \\
&= \sum_{i=1}^{n} \frac{n}{n-(i-1)} \\
&= n \sum_{i=1}^{n} \frac{1}{n-i+1} \\
&= nH_n.
\end{aligned}$$

Since $H_n = \Theta(\log n)$, we get $\mathbb{E}[X] = \Theta(n \log n)$. □

8 High Probability Bound

The following was given in class as an exercise.

Exercise 1. Show that

$$\mathbb{P}(\text{any slot is accessed} \leqslant c \log n \text{ times}) \leqslant \frac{1}{n^2}.$$

Solution.

The expectation of the number of queries is $\Theta(n \log n)$. We now show that $3n \log n$ queries are enough with high probability. Here log is the natural logarithm.

Theorem 3

If random sampling makes $3n \log n$ queries, then

$$\mathbb{P}(\text{fail}) \leqslant \frac{1}{n^2}, \qquad \text{i.e.,} \qquad \mathbb{P}(\text{Success}) \geqslant 1 - \frac{1}{n^2},$$

where "fail" means that some slot is not accessed.

Proof. Let B_i be the event that slot i is not accessed. A query does not access slot i with probability $1 - \frac{1}{n}$. Using $1 - x \leqslant e^{-x}$,

$$\begin{aligned}
\mathbb{P}(B_i) = \mathbb{P}(\text{slot } i \text{ not accessed}) &\leqslant \left(1 - \frac{1}{n}\right)^{3n \log n} \\
&\leqslant e^{-\frac{1}{n} \cdot 3n \log n} \\
&= e^{-3 \ln n} \\
&= \frac{1}{n^3}.
\end{aligned}$$

By the union bound,

$$\mathbb{P}\left(\bigcup_i B_i\right) \leqslant \sum_{i=1}^{n} \mathbb{P}(B_i) = \frac{n}{n^3} = \frac{1}{n^2}.$$

□

9 Lower Bound for Randomized Algorithms

Theorem 4

Any Las Vegas randomized algorithm for search needs $\Omega(n)$ expected queries.

The lower bound is shown using Yao's minmax principle.

Yao's Minmax Principle

Let A^{rand} be a randomized algorithm, and let $\mathbb{E}\left[A^{\text{rand}}\right]$ be its expected number of queries. Let μ be a probability distribution over the inputs, and let D_μ^{best} be the best deterministic algorithm for inputs that are chosen according to μ. Then

$$\mathbb{E}\left[A^{\text{rand}}\right] \geqslant \max_\mu \mathbb{E}_\mu\left[D_\mu^{\text{best}}\right] \geqslant \Omega(n).$$

The quantity $\mathbb{E}_\mu\left[D_\mu^{\text{best}}\right]$ is called the *distributional complexity*.

10 Chernoff Bounds

Let's say that a coin gives heads (H) with probability $\frac{1}{2}+\varepsilon$ and tails (T) with probability $\frac{1}{2}-\varepsilon$. If we toss the coin n times, then

$$\mathbb{E}[\text{Heads}] = n\left(\frac{1}{2}+\varepsilon\right) = \frac{n}{2}+n\varepsilon.$$

Chernoff bounds show that the number of heads is close to its expectation, so that

$$\mathbb{P}\left(\text{Heads} < \frac{n}{2}\right) \sim \text{very small}.$$

In general, let

$$X = \sum_{i=1}^n X_i, \qquad X_i = \begin{cases} 1 & \text{with probability } p_i, \\ 0 & \text{with probability } 1-p_i, \end{cases}$$

where the X_i are independent Bernoulli random variables. If $p_i = p$ for all i, then

$$\mathbb{E}[X] = np, \qquad \mathbb{V}[X] = np(1-p).$$

The distribution of X is concentrated around its mean np (Figure 5).

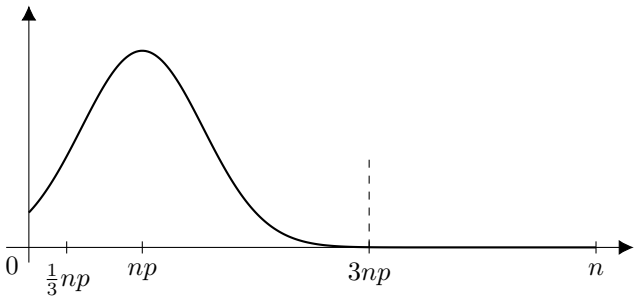


Figure 5: The distribution of X is concentrated around the mean np.

Theorem 5 ▶ Chernoff Bounds

Let $X = \sum_{i=1}^{n} X_i$ be as above and let $\mu = \mathbb{E}[X]$.

- Upper tail (UI): for $0 < \delta < 1$,

$$\mathbb{P}(X \geqslant (1+\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{3}}.$$

- For $R \geqslant 6\mu$,

$$\mathbb{P}(X \geqslant R) \leqslant \frac{1}{2^R}.$$

- Lower tail (LI): for $0 < \delta < 1$,

$$\mathbb{P}(X \leqslant (1-\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}}.$$

Lecture Date: September 3, 2026

Lecture Topic: Markov, Chebyshev and Chernoff Inequalities, Search, Birthday Paradox

In this lecture, we state and prove Markov's inequality and Chebyshev's inequality, and we state the Chernoff bound. We then look at the query complexity of search for classical and quantum algorithms, and we analyze the birthday paradox.

11 Markov's Inequality

Markov's inequality uses only the expectation $\mathbb{E}[X]$ of the random variable, which is also called the *first moment*.

Theorem 6 ► Markov's Inequality

Let $X \geqslant 0$ be a random variable. Then, for any $a > 0$,

$$\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}.$$

Equivalently, for any $b > 0$,

$$\mathbb{P}(X \geqslant b) \leqslant \frac{\mathbb{E}[X]}{b}.$$

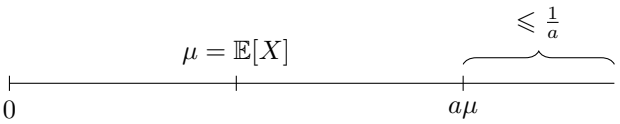


Figure 6: Markov's inequality: the probability that X is at least $a\mu$ is at most $1/a$.

Proof. Let

$$I = \begin{cases} 1 & \text{if } X \geqslant b, \\ 0 & \text{otherwise.} \end{cases}$$

Then

$$I \leqslant \frac{X}{b}.$$

This is true for $X \geqslant b$, since then $I = 1 \leqslant \frac{X}{b}$. It is also true for $X < b$, since then $I = 0$ and $X \geqslant 0$. Markov's inequality follows by taking expectations on both sides:

$$\mathbb{E}[I] \leqslant \mathbb{E}\left[\frac{X}{b}\right] = \frac{\mathbb{E}[X]}{b}.$$

Since

$$\mathbb{P}(I = 1) = \mathbb{P}(X \geqslant b) \qquad \text{and} \qquad \mathbb{P}(I = 0) = \mathbb{P}(X < b),$$

we have $\mathbb{E}[I] = \mathbb{P}(I = 1)$, and hence

$$\mathbb{P}(X \geqslant b) \leqslant \frac{\mathbb{E}[X]}{b}.$$

Taking $b = a\,\mathbb{E}[X]$ gives $\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}$. □

12 Chebyshev's Inequality

Chebyshev's inequality uses the variance of the random variable, which is also called the *second moment*.

Definition 5 ▶ Variance and Standard Deviation

The **variance** of a random variable X is

$$\mathbb{V}[X] = \mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big].$$

The **standard deviation** of X is

$$\sigma = \sqrt{\mathbb{V}[X]}.$$

Theorem 7 ▶ Chebyshev's Inequality

For any random variable X and any $a > 0$,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) \leqslant \frac{\mathbb{V}[X]}{a^2} = \frac{\sigma^2}{a^2}.$$

Here $|X - \mathbb{E}[X]|$ is the deviation of X from its mean.

Proof.

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) = \mathbb{P}\Big((X - \mathbb{E}[X])^2 \geqslant a^2\Big) \leqslant \frac{\mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big]}{a^2} = \frac{\mathbb{V}[X]}{a^2},$$

where the inequality is Markov's inequality applied to the random variable $(X - \mathbb{E}[X])^2 \geqslant 0$. □

13 Chernoff Bound

Let

$$X = \sum_{i=1}^{n} X_i, \qquad X_i = \begin{cases} 1 & \text{with probability } p_i, \\ 0 & \text{with probability } 1 - p_i, \end{cases}$$

where the X_i are *mutually independent*. By linearity of expectation,

$$\mathbb{E}[X] = \mu = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} p_i.$$

Theorem 8 ▶ Chernoff Bound

Let X and μ be as above. Then

- for $0 < \delta < 1$,
$$\mathbb{P}(X \geqslant (1+\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{3}};$$
- for $R \geqslant 6\mu$,
$$\mathbb{P}(X \geqslant R) \leqslant 2^{-R};$$
- for $0 < \delta < 1$,
$$\mathbb{P}(X < (1-\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}}.$$

14 Search

Let $f\colon D \to R$ be a function from a domain D to a range R (Figure 7). Given y, we ask: is there an x such that $f(x) = y$? We consider

$$f\colon [1, \dots, N] \to [1, \dots, N],$$

and we want to find an x such that $f(x) = k$. The function can be seen as an array with n slots, where n is the size of the domain, and the cost of an algorithm is the number of queries to f.

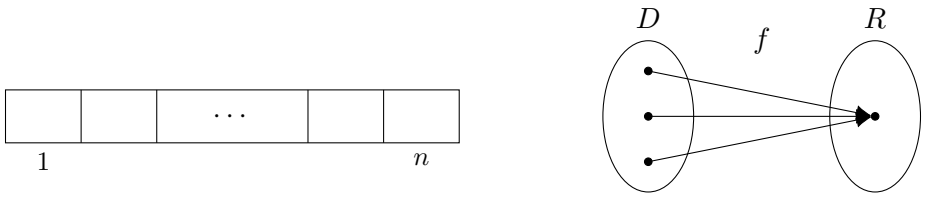


Figure 7: Left: an array with n slots. Right: a function $f\colon D \to R$.

- Randomized classical algorithm: $\Theta(n)$ queries.
- Grover's algorithm: $\mathcal{O}(\sqrt{n})$ queries.

For the random sampling algorithm,

$$\mathbb{E}[\text{queries}] = \mathcal{O}(n), \qquad \mathbb{P}(\text{Success}) = \frac{1}{n}.$$

Probability amplitude amplification. If the classical success probability is p, then in the quantum case it becomes $\sqrt{p}$.

15 Birthday Paradox

There are m people and n birthdays ($n = 365$). We are looking for a *collision*, i.e., $x \neq y$ such that

$$f(x) = f(y),$$

where $f(x)$ is the birthday of person x.

Let E_i be the event that the ith birthday is different from the first $i-1$ birthdays. Then

$$\begin{aligned}
\mathbb{P}\left(\bigcap_{i=1}^{m} E_i\right) &= \mathbb{P}(\text{all birthdays different}) \\
&= \mathbb{P}(E_1) \cdot \mathbb{P}(E_2 \mid E_1) \cdot \mathbb{P}(E_3 \mid E_2 \cap E_1) \cdots \\
&= 1 \cdot \left(1 - \frac{1}{n}\right) \cdot \left(1 - \frac{2}{n}\right) \cdots \left(1 - \frac{m-1}{n}\right) \\
&= \prod_{i=0}^{m-1} \left(1 - \frac{i}{n}\right) \\
&\leqslant \prod_{i=0}^{m-1} e^{-i/n} \\
&= e^{-\sum_{i=0}^{m-1} \frac{i}{n}} \\
&= e^{-\frac{(m-1)m}{2n}}.
\end{aligned}$$

For $m = \sqrt{2n} + 1$,

$$\mathbb{P}(\text{all birthdays different}) \leqslant e^{-\frac{\sqrt{2n}(\sqrt{2n}+1)}{2n}} \leqslant e^{-\frac{2n}{2n}} = \frac{1}{e} \leqslant \frac{1}{2}.$$

Hence $\mathcal{O}(\sqrt{n})$ people are enough to have a constant probability of collision.

15.1 Expected Number of Collisions

The birthday paradox can be seen as throwing m balls into n bins (Figure 8). We show that for $m \approx \sqrt{2n}$,

$$\mathbb{E}[\#\text{ collisions}] \geqslant 1.$$

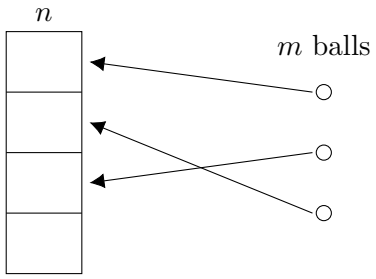


Figure 8: m balls are thrown into n bins.

Let X be the number of collisions. X takes values from 0 to $\binom{m}{2}$. Write

$$X = \sum_{i \neq j} X_{ij}, \qquad X_{ij} = \begin{cases} 1 & \text{if } f(i) = f(j), \\ 0 & \text{else,} \end{cases}$$

where the sum is over the $\binom{m}{2}$ pairs $i \neq j$. The random variables X_{ij} are *pairwise independent*. By linearity of expectation,

$$\begin{aligned} \mathbb{E}[X] &= \sum_{i \neq j} \mathbb{E}[X_{ij}] \\ &= \sum_{i \neq j} \mathbb{P}(X_{ij} = 1) \\ &= \sum_{i \neq j} \frac{1}{n} \\ &= \binom{m}{2} \frac{1}{n} = \frac{m(m-1)}{2n} \sim \frac{m^2}{n}. \end{aligned}$$

Exercise 2. Show that

$$\mathbb{P}(X \geqslant 1) \geqslant \Theta(1),$$

i.e., the birthday paradox occurs.

From class:

$$\mathbb{V}[X] = \mathbb{V}\left[\sum_{i \neq j} X_{ij}\right].$$

Solution.

16 Quantum Birthday Paradox

An algorithm makes k queries

$$f(x_1), \ldots, f(x_k).$$

Exercise 3. Show a $\sqrt{n}$ lower bound in the classical case.

Solution.

Lecture Date: September 24, 2026

Lecture Topic: Deutsch's Algorithm, Phase Flip, Walsh–Hadamard Transform, Deutsch–Jozsa Algorithm

In this lecture, we see the first quantum algorithms that beat every classical algorithm in the query model. We study Deutsch's algorithm, which decides with a single query whether a function $f\colon \{0,1\} \to \{0,1\}$ is constant or balanced, the *phase flip* form of a quantum query, the n-qubit Hadamard (Walsh–Hadamard) transform, and the Deutsch–Jozsa algorithm, which does the same for $f\colon \{0,1\}^n \to \{0,1\}$, again with a single query.

17 Deutsch's Algorithm

Problem 1 ▶ Deutsch's Problem

Given a function $f\colon \{0,1\} \to \{0,1\}$ as a black box, determine whether f is *constant* ($f(0) = f(1)$) or *balanced* ($f(0) \neq f(1)$).

Classically we need 2 queries: after one query, say $f(0)$, both answers are still possible. Quantumly, Deutsch's algorithm does it with 1 query.

17.1 The Quantum Query U_f

A quantum circuit is reversible, but f need not be (a constant f forgets its input). We therefore convert f into a reversible *quantum version* U_f: a controlled-f gate modeled on the CNOT gate (Figure 9). CNOT maps $|a\rangle|b\rangle \mapsto |a\rangle|a \oplus b\rangle$, and U_f maps

$$U_f\colon |a\rangle|b\rangle \longmapsto |a\rangle|b \oplus f(a)\rangle.$$

The first qubit (the input a) is left unchanged, and $f(a)$ is XORed into the second qubit. Applying U_f twice gives back $|a\rangle|b\rangle$, so U_f is reversible. One application of U_f counts as one query to f.

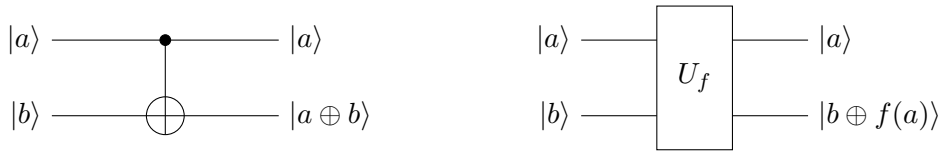


Figure 9: Left: the CNOT gate. Right: the quantum query U_f, a controlled-f gate.

17.2 The Circuit

Deutsch's algorithm is usually given in the form of Figure 10: the first qubit starts in $|0\rangle$ and the second in $|1\rangle$; both go through a Hadamard gate, then through U_f, then through a Hadamard gate again, and are measured. We will show that the measurement of the first qubit gives $|0\rangle$ if f is constant and $|1\rangle$ if f is balanced (the second qubit always gives $|1\rangle$).

Recall that the Hadamard gate H acts as

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

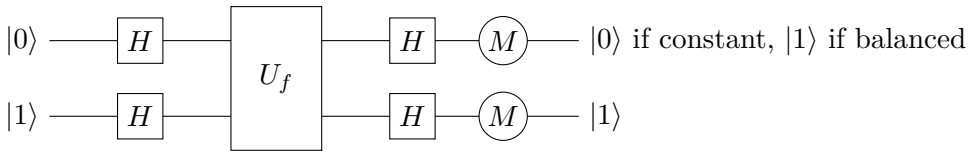


Figure 10: Deutsch's algorithm. It makes one query (U_f); the first measurement decides whether f is constant or balanced.

or, in one formula, for $a \in \{0,1\}$,

$$H|a\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle) = \frac{1}{\sqrt{2}} \sum_{b\in\{0,1\}} (-1)^{ab}|b\rangle. \tag{1}$$

Since $H^2 = I$, applying H to the right-hand side of Equation (1) gives back $|a\rangle$.

Theorem 9 ▶ Deutsch's Algorithm

The circuit of Figure 10 makes one query to f, and measuring its first qubit gives $f(0) \oplus f(1)$ with probability 1. Hence the outcome is 0 if f is constant and 1 if f is balanced.

Proof. After the first two Hadamard gates the state is

$$\begin{aligned} |\psi_1\rangle = H|0\rangle \otimes H|1\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ &= \frac{1}{2}(|0\rangle(|0\rangle - |1\rangle) + |1\rangle(|0\rangle - |1\rangle)). \end{aligned}$$

Applying U_f to each term, and using that $|0 \oplus c\rangle - |1 \oplus c\rangle = (-1)^c(|0\rangle - |1\rangle)$ for $c \in \{0,1\}$ (for $c = 1$ the two kets are swapped),

$$\begin{aligned} |\psi_2\rangle = U_f|\psi_1\rangle &= \frac{1}{2}(|0\rangle(|0 \oplus f(0)\rangle - |1 \oplus f(0)\rangle) + |1\rangle(|0 \oplus f(1)\rangle - |1 \oplus f(1)\rangle)) \\ &= \frac{1}{2}\Big(|0\rangle(-1)^{f(0)}(|0\rangle - |1\rangle) + |1\rangle(-1)^{f(1)}(|0\rangle - |1\rangle)\Big) \\ &= \frac{1}{\sqrt{2}}\Big((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\Big) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ &= (-1)^{f(0)}\frac{1}{\sqrt{2}}\Big(|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle\Big) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \end{aligned}$$

where in the last step we used $(-1)^{f(1)} = (-1)^{f(0)}(-1)^{f(0)\oplus f(1)}$. The first qubit is now exactly the right-hand side of Equation (1) with $a = f(0) \oplus f(1)$, up to the sign $(-1)^{f(0)}$. Applying H to it therefore gives

$$|\psi_3\rangle = (-1)^{f(0)}|f(0) \oplus f(1)\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

The factor $(-1)^{f(0)}$ is a global phase and does not affect the measurement, so measuring the first qubit gives $f(0) \oplus f(1)$ with probability 1. (Applying H to the second qubit gives $|1\rangle$, which is why the second measurement in Figure 10 always gives 1.) □

18 Phase Flip

The computation above shows a general trick. If the second (target) qubit of U_f is in the state $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, then for every input x

$$U_f\,|x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = (-1)^{f(x)}\,|x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

i.e., the target qubit is unchanged and the value $f(x)$ shows up as a sign in front of $|x\rangle$. Since the target qubit never changes, we drop it and write the query in the short-hand *phase flip* form

$$U_f\colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle,$$

instead of the original form $U_f\colon |a\rangle|b\rangle \mapsto |a\rangle|b \oplus f(a)\rangle$. By linearity, U_f maps a superposition $\sum_x \alpha_x|x\rangle$ to $\sum_x \alpha_x(-1)^{f(x)}|x\rangle$: one query flips the sign of every $|x\rangle$ with $f(x) = 1$ at once.

19 The n-Qubit Hadamard Transform

To generalize Deutsch's algorithm to n bits we need the Hadamard gate applied to each of n qubits. Start with $n = 2$ and let $|x\rangle = |x_1x_2\rangle = |x_1\rangle|x_2\rangle$. Using Equation (1) on each qubit,

$$\begin{aligned}(H \otimes H)|x\rangle &= H|x_1\rangle \otimes H|x_2\rangle \\ &= \left(\frac{1}{\sqrt{2}}\sum_{y_1\in\{0,1\}}(-1)^{x_1y_1}|y_1\rangle\right)\left(\frac{1}{\sqrt{2}}\sum_{y_2\in\{0,1\}}(-1)^{x_2y_2}|y_2\rangle\right) \\ &= \frac{1}{2}\sum_{y\in\{0,1\}^2}(-1)^{x_1y_1+x_2y_2}|y\rangle = \frac{1}{2}\sum_{y\in\{0,1\}^2}(-1)^{x\cdot y}|y\rangle,\end{aligned}$$

where $x \cdot y = x_1y_1 \oplus x_2y_2$ is the inner product of the bit strings modulo 2 (only the parity of the exponent matters). In general, for $x, y \in \{0,1\}^n$ we write

$$x \cdot y = x_1y_1 \oplus x_2y_2 \oplus \cdots \oplus x_ny_n.$$

Lemma 10 ▶ Walsh–Hadamard Transform

The n-qubit Hadamard transform $H^{\otimes n} = H \otimes H \otimes \cdots \otimes H$ (the *Walsh–Hadamard transform*) acts on a basis state $|x\rangle = |x_1x_2\cdots x_n\rangle$ as

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}}\sum_{y\in\{0,1\}^n}(-1)^{x\cdot y}|y\rangle.$$

Proof. Exactly as for $n = 2$: apply Equation (1) to each of the n qubits and multiply out the n sums. The exponent is $x_1y_1 + \cdots + x_ny_n$, and (-1) to this power only depends on its parity $x \cdot y$. □

The most important special case is $x = 0^n = 00\cdots0$: then $x \cdot y = 0$ for every y, and

$$H^{\otimes n}|00\cdots0\rangle = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle$$

is the uniform superposition of all 2^n bit strings.

20 Deutsch–Jozsa Algorithm

Problem 2 ▶ Deutsch–Jozsa Problem

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, with the *promise* that f is either constant or balanced (i.e., $f(x) = 0$ for exactly half of the 2^n inputs), determine which is the case.

This is the same problem as Deutsch's, but on n bits. It is a *promise problem*: we are told in advance that f is one of the two kinds. Let $N = 2^n$ be the number of inputs.

20.1 Classical Query Complexity

- *Deterministic:* in the worst case $2^{n-1}+1 = N/2+1$ queries are needed. If an algorithm has seen only $N/2$ values and they are all equal, then f may still be either constant or balanced. The same holds for a randomized algorithm that has to succeed with probability 1: in the worst case it needs $2^{n-1}+1$ queries.
- *Randomized (Monte Carlo):* query f on α independent uniformly random inputs. If two of the answers differ, output "balanced"; otherwise output "constant". If f is constant, we are always right. If f is balanced, every answer is 0 or 1 with probability $1/2$ each, so all α answers agree, and we fail, with probability

$$2 \cdot \left(\frac{1}{2}\right)^{\alpha} = \left(\frac{1}{2}\right)^{\alpha-1}.$$

  A constant number of queries gives a constant error probability, and $\alpha = \mathcal{O}(\log N)$ queries bring the error probability down to $1/N$.
- *Quantum:* the Deutsch–Jozsa algorithm needs 1 query (and n measurements) and makes no error.

20.2 The Algorithm

The circuit (Figure 11) is Deutsch's circuit with n input qubits: n qubits in $|0\rangle$ and one qubit in $|1\rangle$ go through Hadamard gates, then through U_f (one query), then the n input qubits go through Hadamard gates again and are measured. If any of the n measurements gives 1, then f is balanced; if they all give 0, then f is constant.

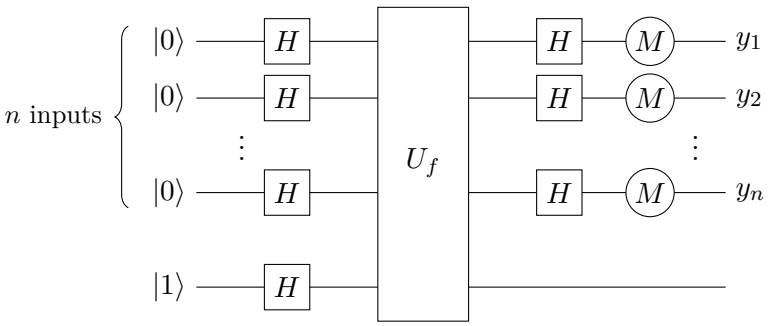


Figure 11: The Deutsch–Jozsa algorithm. It makes one query and n measurements; the last qubit is not needed after the query and is not measured.

Theorem 11 ▶ Deutsch–Jozsa Algorithm

In the circuit of Figure 11, measuring the n input qubits gives $y = 0^n$ with probability 1 if f is constant, and with probability 0 if f is balanced. Hence, if any measurement gives 1, then f is balanced; otherwise f is constant.

Proof. We use the phase flip form of U_f: the last qubit is $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ throughout and is dropped. The n input qubits start in $|00\cdots0\rangle$. After the first layer of Hadamard gates,

$$H^{\otimes n}|00\cdots0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x\rangle.$$

The query U_f flips the sign of every $|x\rangle$ with $f(x) = 1$:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle.$$

After the second layer of Hadamard gates, by Lemma 10,

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle = \frac{1}{2^n} \sum_{y \in \{0,1\}^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+x \cdot y} |y\rangle.$$

Look at the amplitude of $|y\rangle$ for $y = 0^n$. Since $x \cdot 0^n = 0$ for every x, it is

$$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} = \begin{cases} \pm 1 & \text{if } f \text{ is constant (all } 2^n \text{ terms are equal),} \\ 0 & \text{if } f \text{ is balanced (half the terms are } +1\text{, half are } -1\text{).} \end{cases}$$

If f is constant, the state is $\pm|00\cdots0\rangle$ (all other amplitudes must be 0, since the amplitudes squared sum to 1), so the measurement gives 0^n with probability 1. If f is balanced, the amplitude of $|00\cdots0\rangle$ is 0, so the measurement gives 0^n with probability 0, i.e., at least one of the n measurements gives 1. □

For $n = 1$ this is exactly Deutsch's algorithm. In both cases a single query suffices, whereas any classical algorithm with zero error needs $2^{n-1} + 1$ queries.